Module Guide for Software Engineering

Team 8, AlgoCatan
Jake Read
Rebecca Di Filippo
Matthew Cheung
Sunny Yao

January 19th, 2026

1 Revision History

Date	Version	Notes
11/13/2025	1.0	Rev -1
01/14/2026	1.1	Rev 0
04/06/2026	1.2	Final Doc Review

2 Reference Material

This section records information for easy reference.

2.1 Abbreviations and Acronyms

symbol	description
AC	Anticipated Change
AI	Artificial Intelligence
LLM	Large Language Model
CV	Computer Vision
DAG	Directed Acyclic Graph
FR	Functional Requirement
M	Module
MG	Module Guide
NFR	Non-Functional Requirement
OS	Operating System
R	Requirement
RL	Reinforcement Learning
SRS	Software Requirements Specification
UC	Unlikely Change
UI	User Interface
VnV	Verification and Validation

Contents

List of Tables

List of Figures

3 Introduction

Decomposing a system into modules is a commonly accepted approach to developing software. A module is a work assignment for a programmer or programming team (Parnas et al., 1984). We advocate a decomposition based on the principle of information hiding (Parnas, 1972). This principle supports design for change, because the "secrets" that each module hides represent likely future changes. Our design follows the rules layed out by Parnas et al. (1984), as follows:

- System details that are likely to change independently should be the secrets of separate modules.
- Each data structure is implemented in only one module.
- Any other program that requires information stored in a module's data structures must obtain it by calling access programs belonging to that module.

After completing the first stage of the design, the Software Requirements Specification (SRS), the Module Guide (MG) is developed (Parnas et al., 1984). The MG specifies the modular structure of the system and is intended to allow both designers and maintainers to easily identify the parts of the software. The rest of the document is organized as follows. Section 4 lists the anticipated and unlikely changes of the software requirements. Section 5 summarizes the module decomposition that was constructed according to the likely changes. Section 6 specifies the connections between the software requirements and the modules. Section 7 gives a detailed description of the modules. Section 8 includes two traceability matrices. One checks the completeness of the design against the requirements provided in the SRS. The other shows the relation between anticipated changes and the modules. Section 9 describes the use relation between modules.

4 Anticipated and Unlikely Changes

This section lists possible changes to the system. According to the likeliness of the change, the possible changes are classified into two categories. Anticipated changes are listed in Section 4.1, and unlikely changes are listed in Section 4.2.

4.1 Anticipated Changes

Anticipated changes are the source of the information that is to be hidden inside the modules. Ideally, changing one of the anticipated changes will only require changing the one module that hides the associated decision.

AC1: The specific architecture, heuristics, and reward functions of the RL agent. These will be iterated on constantly to improve performance.

AC2: The specific CV algorithm or model (e.g., YOLOv9 vs. OpenCV) used for board state detection (Deferred). This may be changed to improve accuracy or adapt to different lighting conditions.

AC3: The layout and design of the User Interface. This will likely change based on user feedback to improve usability.

AC4: The underlying Catan simulator (Catanatron). This is an external library that may be updated or replaced.

AC5: The specific schema or technology (e.g., SQL vs NoSQL) for the Game State Database, as post-game analysis needs may evolve.

AC6: The communication protocol (e.g., 0MQ) used by the Image Queue. This might be swapped for another low-latency middleware like WebSockets (Deferred).

4.2 Unlikely Changes

The module design should be as general as possible. However, fixing some design decisions at the system architecture stage can simplify the software design. It is not intended that these decisions will be changed.

UC1: The fundamental rules of the standard Settlers of Catan base game. The project is not intended to support expansions.

UC2: The core architectural decomposition. The system will always require separate components for vision, state management, AI, and a user interface.

UC3: The primary technology stack choices of Python for the backend/AI and JavaScript/React for the frontend.

UC4: The fundamental premise of reading from a physical game board via a camera, as opposed to being a purely digital application (Deferred).

5 Module Hierarchy

This section provides an overview of the module design. Modules are summarized in a hierarchy decomposed by secrets in Table 1. The modules listed below, which are leaves in the hierarchy tree, are the modules that will actually be implemented.

M1: Hardware-Hiding Module (OS) (Deferred)

M2: Computer Vision Model (Deferred)

M3: User Interface

M4: Game State Manager

M5: Training Pipeline

M6: AI Model

M7: Game State Database

M8: Backend API Server

M9: Elo League System

M10: Curriculum Learning Manager

M11: Explanation Pipeline

Table 1: Module Hierarchy

Level 1	**Level 2**
Hardware-Hiding Module	M1: Hardware-Hiding Module (OS) M2: Computer Vision Model
Behaviour-Hiding Module	M3: User Interface M4: Game State Manager M5: Training Pipeline
Software Decision Module	M6: AI Model M7: Game State Database M8: Backend API Server M9: Elo League System M10: Curriculum Learning Manager M11: Explanation Pipeline

6 Connection Between Requirements and Design

The design of the system is intended to satisfy the requirements developed in the SRS. The following key design decisions were made to address specific requirements:

- **Training Pipeline (M5):** To satisfy FR.S.2.x and support FR.Sa.12, a dedicated training pipeline coordinates self-play, reward collection, evaluation, checkpointing, and integration with both curriculum phases and league-based opponent selection.

- **AI Model (M6):** To satisfy FR.S.3.1–FR.S.3.5, the AI model module encapsulates inference logic, model weights, and move selection behaviour, allowing model versions to be swapped independently of the rest of the system.
- **Elo League System (M9):** To satisfy FR.S.3.6–FR.S.3.10, a dedicated league module maintains ratings for trained snapshots and baseline bots, supports opponent selection, and manages pruning/storage policies.
- **Curriculum Learning Manager (M10):** To satisfy FR.S.3.12–FR.S.3.16, curriculum phase definitions and advancement rules are isolated in their own module so that training complexity can evolve without changing unrelated system logic.
- **User Interface (M3):** To satisfy FR.S.4.x and NFR.S.2, the web frontend is isolated as its own module, enabling independent iteration of gameplay presentation, replay views, bot selection, and explanation display.
- **Game State Database (M7):** To satisfy FR.S.5.x and NFR.S.5, a dedicated database module manages persistent storage for game history, replay data, model metadata, and league-related records.
- **Backend API Server (M8):** To satisfy FR.S.6.x, NFR.S.1, and NFR.S.6, a dedicated backend communication layer handles requests between the frontend and backend modules, including gameplay, replay, bot selection, and explanation requests.
- **Game State Manager (M4):** To satisfy FR.S.7.x and FR.Sa.2–FR.Sa.3, the authoritative game state and move validation logic are encapsulated in a separate module to preserve consistency across gameplay, replay, and explanation workflows.
- **Explanation Pipeline (M11):** To satisfy FR.S.8.x, a dedicated explanation module formats move/state context, communicates with the external LLM service, and returns user-facing natural-language explanations independently of the core AI inference module.
- **Computer Vision Module (M2):** The CV module is retained only for deferred future scope, preserving traceability to FR.S.1.x and related safety requirements without affecting the present web-based architecture.
- **Hardware-Hiding Module (M1):** This module is also deferred, as the current design focuses on a purely digital implementation. However, its inclusion in the hierarchy allows for future expansion to support physical board input without major architectural changes.

7 Module Decomposition

Modules are decomposed according to the principle of "information hiding" proposed by Parnas et al. (1984). The *Secrets* field in a module decomposition is a brief statement of the

design decision hidden by the module. The *Services* field specifies *what* the module will do without documenting *how* to do it.

7.1 Hardware Hiding Modules

7.1.1 Hardware-Hiding Module (M1)

Secrets: The data structures and algorithms used to implement the underlying virtual hardware and platform services. Camera-specific device interaction is only relevant if the deferred CV path is later implemented.

Services: Serves as virtual hardware used by the rest of the system. This module provides the interface between the operating system/platform and the software.

Implemented By: OS

7.1.2 Computer Vision Model (M2)

Secrets: The specific algorithms, libraries (e.g., OpenCV, YOLOv9), and trained models used to interpret a raw video/image feed and detect the physical board layout, pieces, and player actions. Hides the complexity of calibration and error correction.

Services: Provides access programs to process camera input, detect board elements, and translate visual features into a structured digital representation of the game state. Provides diagnostic feedback on detection confidence. This module is deferred and not part of the current implementation.

Implemented By: Python using OpenCV/YOLOv9-based computer vision tooling, trained models, and supporting scripts within Software Engineering

7.2 Behaviour-Hiding Module

Secrets: The contents of the required behaviours.

Services: Includes programs that provide externally visible behaviour of the system as specified in the SRS. This module serves as a communication layer between the user-facing functionality and the underlying software decision modules.

Implemented By: –

7.2.1 User Interface (M3)

Secrets: The specific frontend framework, state presentation logic, UI/UX choices, and replay/explanation interaction patterns used to render the system. Hides the details of DOM manipulation, component composition, and client-side event handling.

Services: Provides an interactive visualization of the *Catan* board and current game state. Allows users to play games, view replays, select opponents, and request/display move explanations.

Implemented By: React/JavaScript frontend.

7.2.2 Game State Manager (M4)

Secrets: The internal data structures and logic used to represent the authoritative game state and enforce the official *Catan* ruleset. Hides synchronization and validation logic across gameplay and replay.

Services: Maintains the single source of truth for the current game state. Tracks player assets, turn data, and legal actions. Provides access programs to update, query, and validate game states and moves.

Implemented By: Integration and Python-based modification of the open-source Catanatron simulator.

7.2.3 Training Pipeline (M5)

Secrets: The orchestration logic for training runs, including simulator integration, checkpointing, evaluation scheduling, metric collection, and coordination with curriculum and league modules.

Services: Runs and manages AI training workflows. Coordinates self-play, reward collection, evaluation, and model checkpoint generation. Provides access programs for launching and monitoring training runs.

Implemented By: Python training scripts and orchestration code.

7.3 Software Decision Module

Secrets: The design decision based on mathematical theorems, physical facts, or programming considerations. The secrets of this module are *not* described in the SRS.

Services: Includes data structures and algorithms used in the system that do not provide direct interaction with the user.

Implemented By: –

7.3.1 AI Model (M6)

Secrets: The specific DRL model architecture, heuristics, feature encoding, and trained weights used to determine actions. Hides the mathematical formulation and implementation details of policy/value inference.

Services: Analyzes a given game state and produces valid actions or action evaluations. Provides access programs for move inference and model version loading.

Implemented By: PPO-based DRL model implementation.

7.3.2 Game State Database (M7)

Secrets: The specific database technology, schema, indexing strategy, and persistence logic used to store and retrieve game history, replay data, model metadata, and league-related records.

Services: Stores and maintains complete and consistent records of game states, match results, replay data, and model metadata. Provides access programs for efficient read/write operations and structured access to historical data.

Implemented By: PostgreSQL database layer.

7.3.3 Backend API Server (M8)

Secrets: The specific API framework, endpoint structure, serialization format, request/response contracts, and error-handling logic used for frontend/backend communication.

Services: Enables reliable communication between the User Interface and backend modules. Provides access programs for gameplay, replay, bot selection, persistence, and explanation requests.

Implemented By: Python backend API layer.

7.3.4 Elo League System (M9)

Secrets: The Elo update rules, baseline sampling policy, snapshot retention strategy, and league pruning logic used to manage model strength estimation and opponent selection.

Services: Maintains ratings for trained model snapshots and baseline bots. Provides access programs to register league members, update ratings, expose available opponents, and select opponents for training or gameplay.

Implemented By: Python-based software engineering.

7.3.5 Curriculum Learning Manager (M10)

Secrets: The curriculum phase definitions, advancement thresholds, cycling rules, and rule-variation configuration used to control training complexity.

Services: Defines and manages curriculum phases for training. Provides access programs to determine the active phase, evaluate advancement conditions, and log phase transitions.

Implemented By: Python-based software engineering.

7.3.6 Explanation Pipeline (M11)

Secrets: The prompt construction logic, context-packet format, provider-specific LLM integration, and response validation/filtering used to generate natural-language move explanations.

Services: Collects the relevant game-state and move context for an explanation request. Formats requests for the external LLM service and returns user-facing natural-language explanations or clear failure responses.

Implemented By: Python-based backend explanation services with external LLM API integration

8 Traceability Matrix

This section shows two traceability matrices: between the modules and the requirements and between the modules and the anticipated changes.

Table 2: Trace Between Requirements and Modules

Requirement(s)	**Modules**
FR.S.1.x (Computer Vision Model, Deferred)	M2
FR.S.2.x (Training Pipeline)	M5, M9, M10
FR.S.3.1–FR.S.3.5 (AI Model)	M6
FR.S.3.6–FR.S.3.10 (Elo League System)	M9, M7
FR.S.3.12–FR.S.3.16 (Curriculum Learning Manager)	M10, M5
FR.S.4.x (User Interface)	M3, M9, M11
FR.S.5.x (Game State DB)	M7
FR.S.6.x (Backend API Server)	M8
FR.S.7.x (Game State Manager)	M4
FR.S.8.x, FR.Sa.13 (Explanation Pipeline)	M11, M8
NFR.S.1 (Scalability)	M6, M7, M8, M11
NFR.S.2 (Usability)	M3, M11
NFR.S.3 (Maintainability)	M6, M2, M4, M3, M5, M8, M9, M10, M11
NFR.S.4 (Installability)	M1, M8, M3
NFR.S.5 (Data Integrity)	M7, M4, M8
NFR.S.6 (Availability)	M8, M7, M11
FR.Sa.1, FR.Sa.8, FR.Sa.11 (Deferred CV Safety)	M2, M3
FR.Sa.2 (State Resynchronization)	M4, M3
FR.Sa.3 (AI Validation)	M4, M6
FR.Sa.4 (Graceful Failure Messaging)	M3, M8, M11
FR.Sa.5 (DB Safety)	M7
FR.Sa.6, NFR.Sa.2 (Connection Safety)	M8
FR.Sa.7 (Suggestion Visibility)	M3
FR.Sa.9 (Incomplete Data Warning)	M3, M11, M2
FR.Sa.10 (AI Rollback)	M6, M9
FR.Sa.12, 14, 15 (Training Validation)	M5, M9, M10

Table 3: Trace Between Anticipated Changes and Modules

Anticipated Change (AC)	Modules
AC1	M6
AC2	M2
AC3	M3
AC4	M5
AC5	M7
AC6	M8

Table 4: Trace Between Unlikely Changes and Modules

Unlikely Change (UC)	Modules
UC1	M5

9 Use Hierarchy Between Modules

In this section, the uses hierarchy between modules is provided. Module A uses a module B if the correct execution of A depends upon the availability of a correct implementation of B.

For the current system, runtime dependencies are centered around the Backend API Server (M8), while training-side dependencies are centered around the Training Pipeline (M5).

At runtime, the User Interface (M3) depends on the Backend API Server (M8), which in turn depends on the Game State Manager (M4), Game State Database (M7), AI Model (M6), Elo League System (M9), and Explanation Pipeline (M11) depending on the request type.

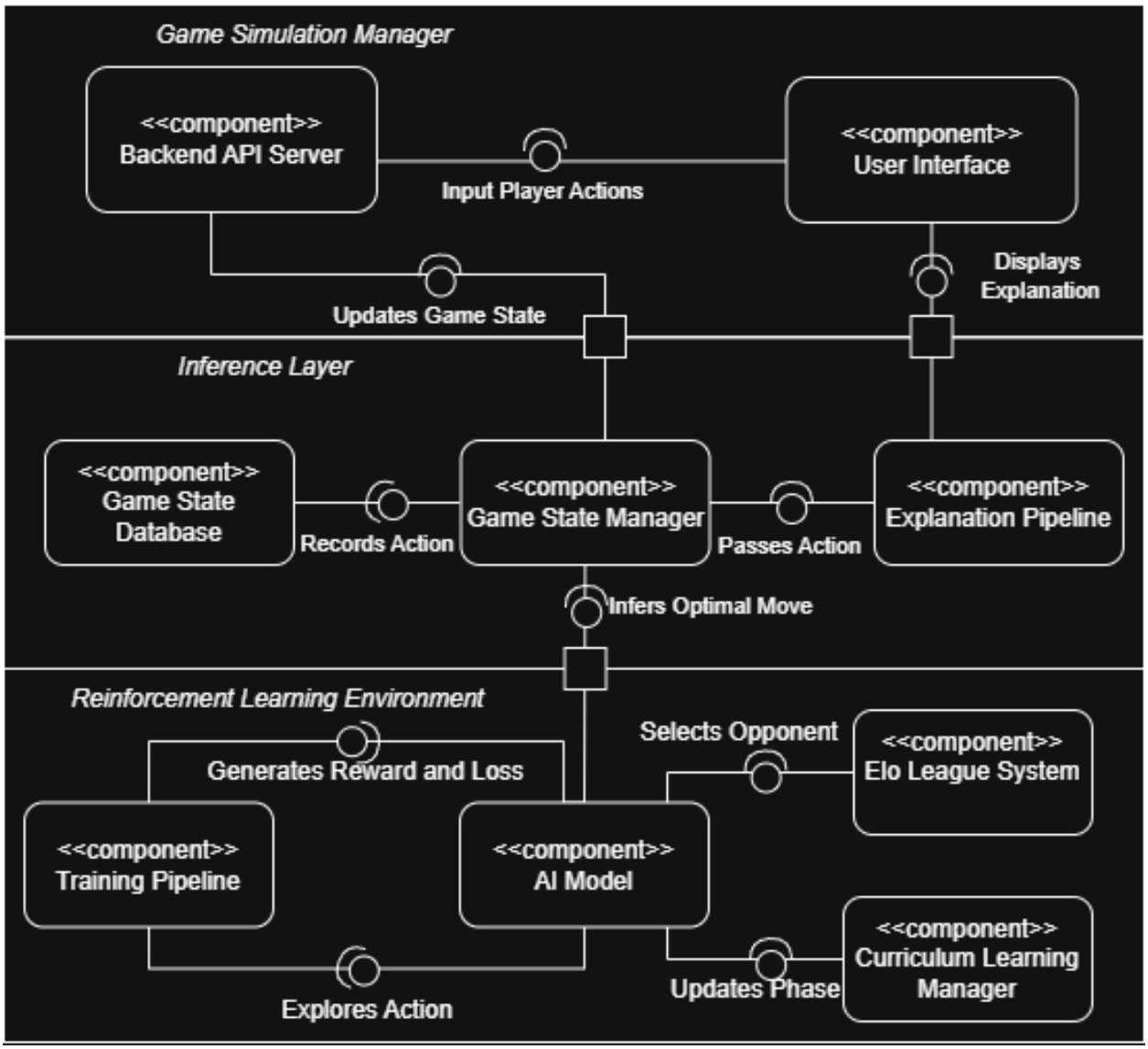

Figure 1: Component diagram showing use hierarchy among modules.

10 User Interfaces

The detailed design of the User Interface (M3) will be designed in stages, and updated following user feedback. As such, the specific UI design is not included in this document. (The usability testing doc in the extras folder covers the iteration process for the UI design in more detail.) The design will focus on satisfying NFR.S.2 (Usability) and the specific UI-related safety requirements (e.g., FR.Sa.1, FR.Sa.7).

11 Design of Communication Protocols

The system's runtime modules communicate primarily through the Backend API Server (M8). In the implemented system, this communication is based on REST-style HTTP requests carrying JSON payloads.

The frontend uses the backend base URL specified by the `CTRON_API_URL` environment variable, defaulting to `http://localhost:5001` in local development. The backend is implemented as a Flask application with CORS enabled, and its API routes are exposed through a blueprint mounted under the `/api` prefix.

Runtime API Contracts

- `POST /api/analytics/start`
  Called by the frontend when a user starts interacting with the application. Logs a homepage/game-start analytics event and returns whether the event was recorded successfully.

- `GET /api/admin/user-starts?limit=<n>`
  Used by the admin page to retrieve recently logged user-start events. Returns a JSON array of rows containing `id`, `ip`, and `timestamp` fields.

- `GET /api/bots`
  Used by the bot-selection pages to retrieve available opponents. Returns bot metadata including identifiers, display names, Elo values, and backend player keys.

- `POST /api/games`
  Creates a new game session. The request body includes a `players` array and may also include optional configuration such as `map_template`, `discard_limit`, `vps_to_win`, `friendly_robber`, and `familiarity`. The response returns a generated `game_id`.

- `GET /api/games/{gameId}/states/{stateIndex}`
  Retrieves a serialized game state for the specified game and state index. The special value `latest` may be used instead of a numeric state index.

- `POST /api/games/{gameId}/actions`
  Advances the game by one action. If the request body is empty, the backend treats the request as a bot turn. If the request body contains an action object, the backend parses and executes the supplied human action.

- `GET /api/games/{gameId}/states/{stateIndex}/mcts-analysis`
  Requests analysis data for a particular stored state. Returns JSON containing a success flag, win-probability estimates, and the analyzed state index.

- `GET /api/games/{gameId}/explain/{moveIndex}`
  Requests an LLM explanation for a specific move in the selected game. Returns JSON containing the queried move index and the generated natural-language explanation.

Frontend Usage of the Protocol

The User Interface (M3) communicates with the Backend API Server (M8) through a typed frontend API client. Bot-selection screens request available bots and create configured games. Gameplay and replay screens request serialized game states, submit actions, request MCTS analysis, and request move explanations when the user selects "Explain Move".

Internal Backend Coordination

Behind these HTTP endpoints, the Backend API Server coordinates with the Game State Manager (M4), Game State Database (M7), AI Model (M6), Elo League System (M9), and Explanation Pipeline (M11) through direct Python calls and shared backend objects. These internal interactions are implementation-level module uses rather than separate externally visible network protocols.

12 Timeline

The schedule of tasks for the implementation of Revision 0 is detailed below. This timeline ensures that the core components are implemented and integrated by the Rev 0 deadline of February 2, 2026. Tasks are assigned to specific team members corresponding to the modules defined in this document. Note that some modules are absent from the table, as they were added post-revision 0, or were deferred at the time of writing.

Table 5: Implementation Timeline for Revision 0

Module	Task Description	Assignee	Due Date
M6	**Advanced Action Masking**: Refine gym wrappers to strictly filter invalid moves to reduce effective state space.	Jake	Jan 18
M6	**Heuristic-Guided Exploration**: Integrate human logic heuristics to guide agent exploration during early training.	Rebecca	Jan 22
M6	**Reward Shaping v2**: Analyze v1 logs to fix behavior loops (e.g., road spamming) and implement complex reward signals.	Jake, Rebecca	Jan 24
M2	**Exploratory Feasibility Study**: Standalone research script to test if board state can be reliably detected from static images.	Matthew	Jan 28
M3	**Agent Visualization**: Adapt CLI visualizer or lightweight web view to display agent metrics (Confidence/Win %) live.	Sunny	Jan 30
M5	**Benchmarking Suite**: Setup automated evaluation against baseline bots to generate win-rate graphs.	Sunny	Jan 31
All	**Rev 0 Training Run**: Execute final 48+ hour training session using finalized v2 rewards and masks.	Team	Feb 1
All	**Rev 0 Verification**: Analyze final model to ensure it beats Random Baseline >95% and competes with heuristic bots.	Team	Feb 2

References

David L. Parnas. On the criteria to be used in decomposing systems into modules. *Comm. ACM*, 15(2):1053–1058, December 1972.

D.L. Parnas, P.C. Clement, and D. M. Weiss. The modular structure of complex systems. In *International Conference on Software Engineering*, pages 408–419, 1984.